

Title	AXI External SRAM Controller Integration Guide
Project	None
Description	AXI External SRAM Controller Integration Guide
Author	holelee
Date	2/Jun/2005

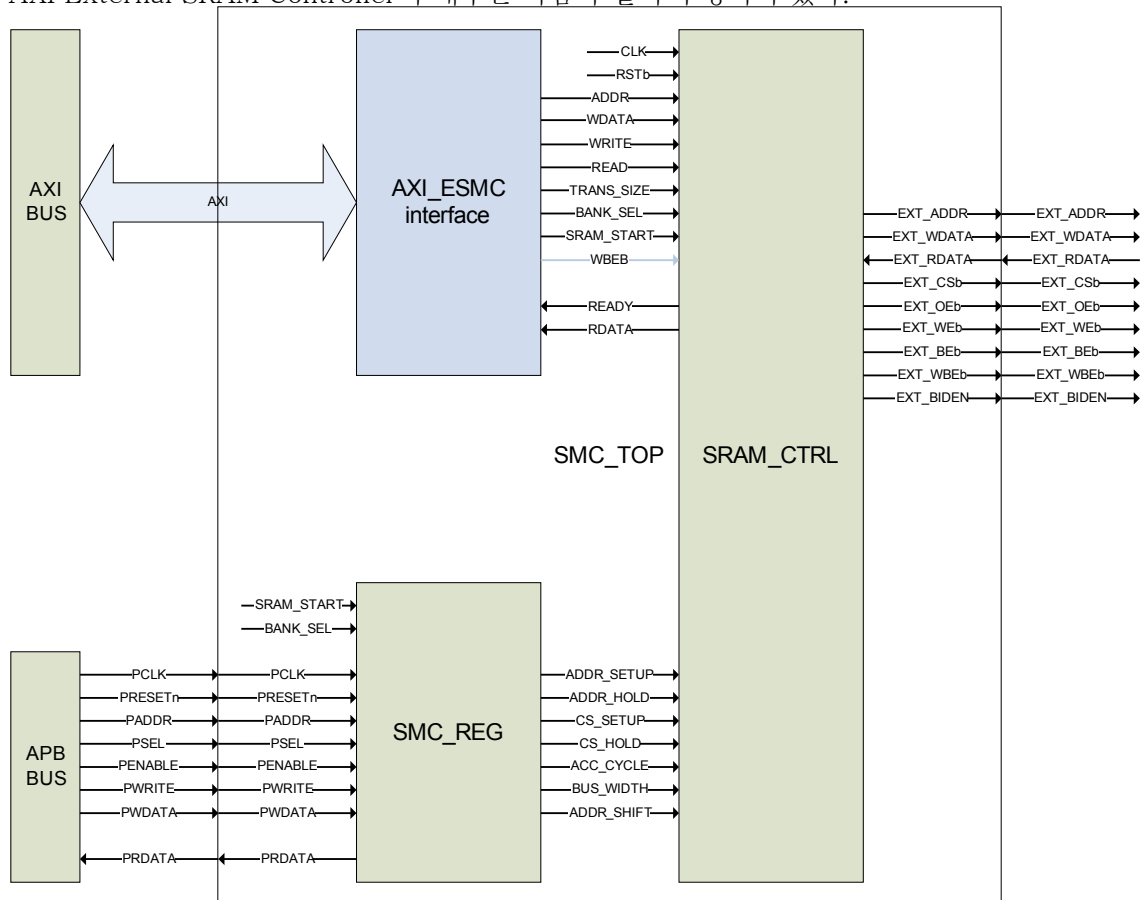
(*) AXI External SRAM Controller의 사용

Asynchronous SRAM interface를 가지는 외부 device를 칩에 연결하기 위해서 사용된다. AXI slave interface를 가지므로 AXI interconnect(BUS)에 붙여서 사용할 수 있다. External SRAM Controller는 ChipSelect time, Address time, Access time을 register setting을 통해서 조정할 수 있다. 현재 설계된 version에서는 READY/nBUSY 출력을 사용하는 variable latency IO device를 붙일 수는 없다.

설계된 AXI External SRAM Controller는 4개의 bank를 가지는데 - 즉 chip select 신호를 4개 가짐 - Verilog code를 수정하여 bank의 숫자를 증가시킬 수 있다.

(*) AXI External SRAM Controller의 구성

AXI External SRAM Controller의 내부는 다음과 같이 구성되어 있다.

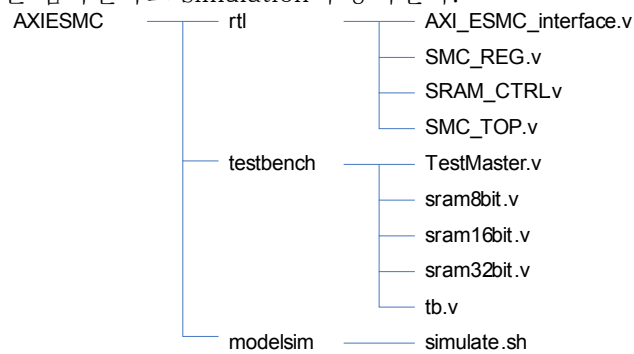


Register setting을 위한 BUS로 APB BUS를 사용하고, Data를 주고 받는 BUS로는 AXI

interface를 사용한다. 내부에 SRAM_CTRL block이 외부 device와 data를 주고 받게 된다.

(*) 제공되는 module 및 파일

기본적으로 AXI External SRAM Controller는 네개의 verilog source code AXI_ESMC_interface.v, SMC_REG.v, SRAM_CTRL.v, SMC_TOP.v로 제공된다. 각 verilog file의 이름은 위에서 알아본 구조의 각 block에 해당하며 SMC_TOP.v가 전체 design의 top module이다. 이 것을 테스트하기 위해서 TestMaster.v, sram8bit.v, sram16bit.v, sram32bit.v, tb.v가 제공되고, shell script인 simulate.sh이 있어서 script를 수행하면 자동으로 modelsim을 이용하여 각 code를 컴파일하고 simulation이 동작한다.



(*) Signals

- AXI interface

AXI slave interface 신호들은 특별한 내용이 없으므로 설명을 생략한다.

- APB 관련

APB BUS 관련 신호들은 특별한 내용이 없으므로 설명을 생략한다.

- 외부 device interface

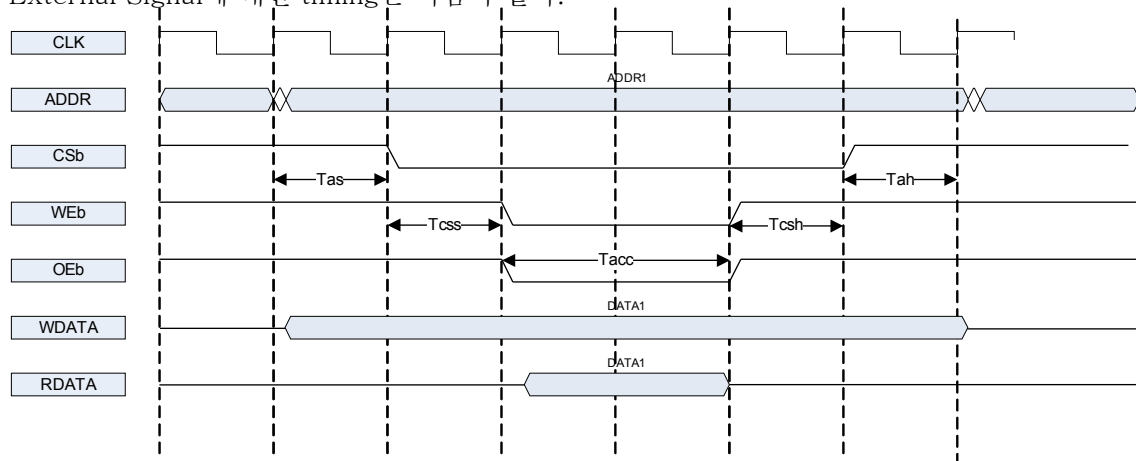
<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
EXT_ADDR[19:0]	OUT	External Device	Address signal
EXT_WDATA[31:0]	OUT	External Device	32bit Write Data
EXT_RDATA[31:0]	IN	External Device	32bit Read Data
EXT_CSb[3:0]	OUT	External Device	ChipSelect signal(active low)
EXT_OEb	OUT	External Device	Output enable signal(active low)
EXT_WEb	OUT	External Device	Write enable signal(active low)
EXT_BEb[3:0]	OUT	External Device	Byte enable signal(active low)
EXT_WBEb[3:0]	OUT	External Device	Write byte enable signal(active low)

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
EXT_BIDEN	OUT	Tristate buffer	Data bus control DATA pins are driven by EXT_WDATA when '1' DATA pins are hi-Z when '0'

참고로 EXT_BEb와 EXT_WBEb는 동일한 신호이지만 timing상에 차이가 있다. EXT_BEb는 ChipSelect 신호(EXT_CSb)와 같은 timing을 가지고 EXT_WBEb는 WriteEanble 신호(EXT_WEB)와 같은 timing을 가지게 된다.

(*) Register

External Signal에 대한 timing은 다음과 같다.



Timing diagram에 있는 time 정보에 대한 약자는 다음과 같다.

Tas : address setup time

Tcss : chip select setup time

Tacc : access time

Tcsh : chip select hold time

Tah : addrese hold time

기본적으로 address 출력이 제일 먼저 이루어 진다. Address setup time이 지난 다음에 CSb가 low가 되고 그 다음 Chip select setup time이 지나면 Output Enable 혹은 Write Enable 신호가 low가 된다. Access time이 지나면 RDATA를 latch하게 되고 그 다음 chip select hold time, address hold time이 지나면 모든 access가 끝나게 된다.

Tas, Tcss, Tacc, Tcsh, Tah를 system clock(정확하게는 AXI interface clock) cycle의 갯수로 정할 수 있는 register가 각 bank 별로 존재한다.

- Register

현재는 bank가 4개 밖에 없으므로 각 bank를 control하는 32bit register가 다음과 같이 존재한다.

<i>Register</i>	<i>Address</i>	<i>R/W</i>	<i>Description</i>	<i>Reset value</i>
smc_bank0	0x00	R/W	Bank0 control	
smc_bank1	0x04	R/W	Bank1 control	
smc_bank2	0x08	R/W	Bank2 control	
smc_bank3	0x0c	R/W	Bank3 control	

<i>smc_bankX</i>	<i>Bit</i>	<i>R/W</i>	<i>Description</i>
reserved	[31:20]	N/A	Reserved
Tas	[19:17]	R/W	Address setup time before CSb falling 000 = 0 clock cycle, 011 = 1 clock cycle 010 = 2 clock cycle, 011 = 3 clock cycle 100 = 4 clock cycle, 101 = 5 clock cycle 110 = 6 clock cycle, 111 = 7 clock cycle
Tcss	[16:14]	R/W	Chip select setup time before OEB/WEB falling 000 = 0 clock cycle, 011 = 1 clock cycle 010 = 2 clock cycle, 011 = 3 clock cycle 100 = 4 clock cycle, 101 = 5 clock cycle 110 = 6 clock cycle, 111 = 7 clock cycle
Tacc	[13:10]	R/W	Access cycle time 0000 = 1 clock cycle, 0001 = 2 clock cycle 0010 = 3 clock cycle, 0011 = 4 clock cycle 0100 = 5 clock cycle, 0101 = 6 clock cycle 0110 = 7 clock cycle, 0111 = 8 clock cycle 1000 = 9 clock cycle, 1001 = 10 clock cycle 1010 = 11 clock cycle, 1011 = 12 clock cycle 1100 = 13 clock cycle, 1101 = 14 clock cycle 1110 = 15 clock cycle, 1111 = 16 clock cycle
Tcsh	[9:7]	R/W	Chip select hold time after OEB/WEB rising 000 = 0 clock cycle, 011 = 1 clock cycle 010 = 2 clock cycle, 011 = 3 clock cycle 100 = 4 clock cycle, 101 = 5 clock cycle 110 = 6 clock cycle, 111 = 7 clock cycle
Tah	[6:4]	R/W	Address hold time after CSb rising 000 = 0 clock cycle, 011 = 1 clock cycle 010 = 2 clock cycle, 011 = 3 clock cycle 100 = 4 clock cycle, 101 = 5 clock cycle 110 = 6 clock cycle, 111 = 7 clock cycle
Width	[3:2]	R/W	Data bus width 00 = 8bit, 01 = 16bit, 10 = 32bit, 11 = reserved

<i>smc_bankX</i>	<i>Bit</i>	<i>R/W</i>	<i>Description</i>
Shift	[1:0]	R/W	Address right shift amount 00 = 0 bit, 01 = 1 bit, 10 = 2bit, 11 = reserved

여기서 Width는 외부에 붙어 있는 SRAM의 data bus의 width를 가르킨다.

shift는 address pin 출력으로 나가는 address와 AXI interface로 받은 address와의 관계를 의미하는데 다음과 같은 관계를 가진다.

$EXT_ADDR = AXI_ADDR \gg shift_amount$

예를 들어 32bit SRAM을 연결하는 경우에는 AXI_ADDR의 최하위 2 bit는 외부 SRAM에서는 무시해야 한다. 이때 사용자는 EXT_ADDR 최하위 2 bit를 사용하지 않고 연결하는 방법도 사용할 수도 있고, 아니면 EXT_ADDR 하위 0비트부터 모두 사용하는 대신에 shift field의 값을 10으로 setting할 수도 있다.

(*) Integration 방법

- configuration 및 사용자가 수정해야 하는 것들

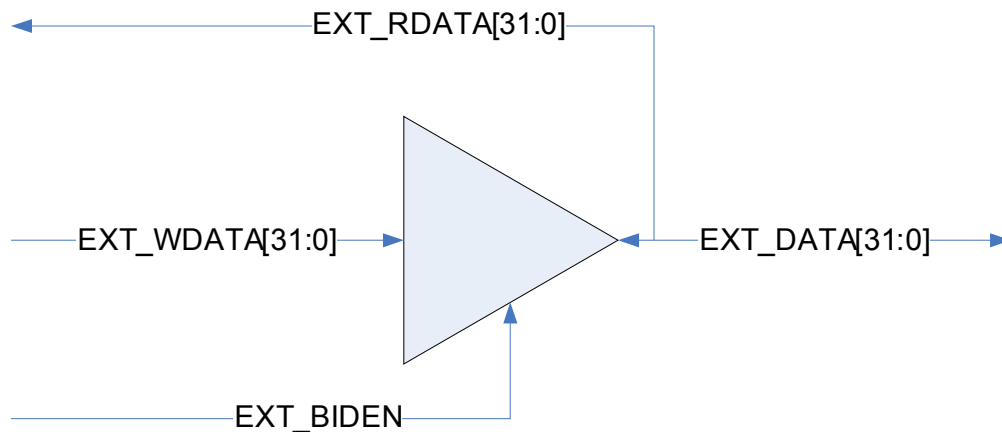
AXI slave interface는 기본적으로 ID width를 configuration할 수 있어야 한다. RID/WID의 bit width는 AXI_ESMC_interface.v에 RID_WIDTH/WID_WIDTH라는 parameter로 존재하는데 그것의 값을 바꾸면 된다.

각 Bank별 address map을 사용자가 만들어야 할 필요가 있다. AXI_ESMC_interface.v의 맨 마지막 부분에 GeneratedAddr이라는 signal을 가지고 SMC_BANK_SEL이라는 신호를 만들어내는 부분이 있다. 현재 설계된 상태는 GeneratedAddr[17:16]을 decoding하고 있는데 이 부분을 적절히 바꾸어서 사용자가 원하는 address map을 가지도록 하면 된다.

Bank의 갯수를 늘리기 위해서는 모든 verilog source code에서 BANK_SIZE를 define한 부분이 있는데 이 것을 변경해 주고, 그에 따라서 변할 수 있는 모든 것을 수정해 주어야 한다. 위에서 언급한 address map에서 SMC_BANK_SEL 생성하는 부분과 SMC_REG.v에 있는 register의 갯수 등을 변경해 주어야 한다.

- port/pin 연결 방법

AXI interface와 APB BUS 관련 신호의 특별한 내용이 없으므로 연결에 대한 설명을 생략한다. 먼저 EXT_RDATA와 EXT_WDATA, EXT_BIDEN은 연결은 다음과 같이 tristate buffer를 사용하여 연결한 후 EXT_DATA를 외부의 device들에 연결하면 된다.



32bit SRAM의 경우 EXT_DATA의 32bit을 모두 사용하면 되고, 16bit SRAM의 경우 EXT_DATA[15:0]만, 8 bit SRAM의 경우 EXT_DATA[7:0]만 연결해 준다.

EXT_ADDR은 0번 bit부터 연결한 후에 각 bank별 shift register를 조정하여도 되고 아니면 SRAM의 bit width에 맞추어 하위 1bit 혹은 2bit를 빼고 연결해도 된다.

EXT_CSb와 EXT_OEb, EXT_WEB는 SRAM의 각 pin에 연결하면 되고, SRAM이 byte enable signal을 가지고 있으면 timing에 맞추어 EXT_BEb 혹은 EXT_WBEb를 연결한다.

여기서 주의할 사항이 하나 있는데, 외부의 device가 ByteEnable pin이 없는 device라면 Write Enable pin에 EXT_WEB를 연결하지 말고 EXT_WBEb를 연결해야 한다. 보통의 8bit SRAM도 byte enable pin이 없으므로 동일하다. AXI Write data channel에서 WSTRB가 모두 0으로 된 signal이 들어올 수 있는데 그런 경우에는 EXT_WEB는 low로 떨어지는데 EXT_WBEb는 high로 유지가 되기 때문이다.

(*) 수정해야 할 사항

범용적으로 사용하기 위해서는 variable latency IO를 지원하도록 수정할 필요가 있다.

현재 설계된 AXI_ESMC_interface의 경우 burst transfer 사이 마다 2~3 cycle의 불필요한 cycle이 들어가 있는데 이 것을 제거할 필요가 있다.